Vulnerability Report

Lesson 5 — Broken Access Control

DVSA Order Workflow — ICS 344 Course Project

- **Course / Section:** ICS 344 — Section 01
- **Student Name(s) + ID(s):** Faisal Alharbi — s202261000
- **Your DVSA Website URL:** http://dvsa-faisalalharbi-ics344-project-716522590330-us-east-1.s3-website.us-east-1.amazonaws.com
- **AWS Region:** us-east-1 (N. Virginia)
- **Date:** 24th April
- **Vulnerability Title(s):** Broken Access Control in DVSA Order Workflow (Lesson 5)

Part 1 — Goal and Vulnerability Summary

This lesson investigated broken access control in the DVSA order workflow. The goal was to determine whether a normal user could reach a privileged backend order-completion path and cause an order to advance outside the intended billing workflow. Testing showed that the public */order* API exposes a complete action that routes to *DVSA-ORDER-COMPLETE* without an *admin* check in *DVSA-ORDER-MANAGER*. However, in my deployed environment, I was not able to reproduce an unauthorized order-state change through this action. The normal billing flow still behaved as expected and moved the order into the processed state.

Part 2 — Why This Works / Root Cause

The underlying security concern in Lesson 5 is weak server-side protection around a sensitive order-management action. In the backend routing logic, *DVSA-ORDER-MANAGER* exposes a public *complete* action that forwards only an *orderId* to *DVSA-ORDER-COMPLETE*, without performing an explicit admin or ownership check in that specific path. This creates a risky trust boundary because a privileged workflow step appears reachable through the public */order* API. In a vulnerable design, that could allow an unprivileged user to trigger a backend state transition outside the intended billing process. In my deployment, the expected unauthorized order-state change did not succeed, but the code still shows inconsistent authorization and workflow enforcement around sensitive order operations, which is why this lesson remains a broken access control investigation rather than a confirmed exploit in my environment.

Part 3 — Environment and Setup

The testing was performed on my own DVSA deployment in AWS in the *us-east-1* region. The lesson focused on the public DVSA order workflow exposed through the website and the backend */dvsa/order* API path used by the frontend. The main AWS components examined during this lesson were API Gateway, the Lambda functions *DVSA-ORDER-MANAGER, DVSA-ORDER-COMPLETE, DVSA-ORDER-BILLING, DVSA-ORDER-GET, DVSA-ADMIN-GET-ORDERS, and DVSA-ADMIN-UPDATE-ORDERS*, and the DynamoDB table *DVSA-ORDERS-DB*. The test context was a normal non-admin DVSA user account. The tools used were the DVSA web interface for normal order creation and checkout, browser Developer Tools to capture requests, headers, and request payloads, the AWS Lambda console to inspect backend routing and function code, and the DynamoDB console to verify order records and status values before and after normal billing. This setup was sufficient to evaluate whether a privileged order-management path was reachable from an unprivileged context.

Part 4 — Reproduction Steps

1. **Logged in to the DVSA website as a normal non-admin user and created a normal order through the store and cart workflow.**
2. **Completed the normal checkout flow once in order to observe the intended behavior of the application. This included adding shipping details, reaching the billing page, submitting valid test card data, and confirming that the order later appeared in My Orders with a processed status and a confirmation token.**
3. **Opened browser Developer Tools and captured the public API path used by the frontend for order operations, the request payloads sent during the normal workflow, and the Authorization header used by the normal user session.**

4. **Inspected the backend code in AWS Lambda and verified that** *DVSA-ORDER-MANAGER* **exposes several public actions on the** */dvsa/order* **path, including** *orders, get, shipping, billing, and complete***.**
5. **Verified in the** *DVSA-ORDER-MANAGER* **source code that the** *case "complete"* **branch forwards only the** *orderId* **to** *DVSA-ORDER-COMPLETE* **using the** *payload { "orderId": req["order-id"] }***, without an explicit admin check in that routing branch.**
6. **Created a second order as the same normal user, but this time stopped before submitting normal billing.**
7. **Opened the order record in DynamoDB and confirmed that the unfinished order remained in the pre-payment state with** *orderStatus = 100***, with no valid completion outcome yet recorded.**
8. **Used the captured normal-user Authorization token to send a direct request to the public** */dvsa/order* **API path with the** *body {"action":"complete","order-id":"REDACTED_ORDER_ID"}* **to test whether the public action could improperly advance the order outside the intended payment workflow.**
9. **Observed that the backend did not change the unpaid order into a paid or processed state. Instead, the response returned an error similar to** *{"status":"err","msg":"order was already processed"}* **rather than producing unauthorized completion.**
10. **Continued the same order through the legitimate billing path in the DVSA web interface and verified that after normal billing the record changed as expected, including a populated confirmation token, total amount, and a later processed state visible in the orders view.**
11. **Reviewed the related backend functions and surrounding code to understand why the observed behavior did not match the originally expected exploit outcome, and concluded that in my deployment the lesson's intended exploit path was not successfully reproducible even though the exposed routing design still appeared weak and security-sensitive.**

Part 5 — Evidence and Proof

The evidence collected during testing showed that the Lesson 5 issue could be investigated from both the frontend request flow and the backend Lambda logic, but the final unauthorized order-state change was not reproduced in my deployment. First, the *DVSA-ORDER-MANAGER* code showed that the public */dvsa/order* API exposes a *complete* action which routes to *DVSA-ORDER-COMPLETE* using only an *orderId* value. This was treated as the main suspicious access-control boundary. Second, the *DVSA-ORDER-COMPLETE* code showed that the function only returns success when the order is already in status *120*, and otherwise returns an error indicating that the order was already processed. Third, the DynamoDB record for the unfinished second order showed *orderStatus = 100* before normal billing, which confirmed that the order had not yet reached a paid or processed state at the time of the direct test. Fourth, the manual direct request to the public */dvsa/order* API using *{"action":"complete","order-id":"..."}* returned an error message instead of advancing the order. Finally, after the legitimate billing step was completed through the normal DVSA workflow, the same order record in DynamoDB showed a later processed state with a populated confirmation token and total amount, and the

order appeared in the **My Orders** page as processed. Together, this evidence showed that the risky public action existed, but the intended broken-access-control outcome was not achieved in my deployed environment.

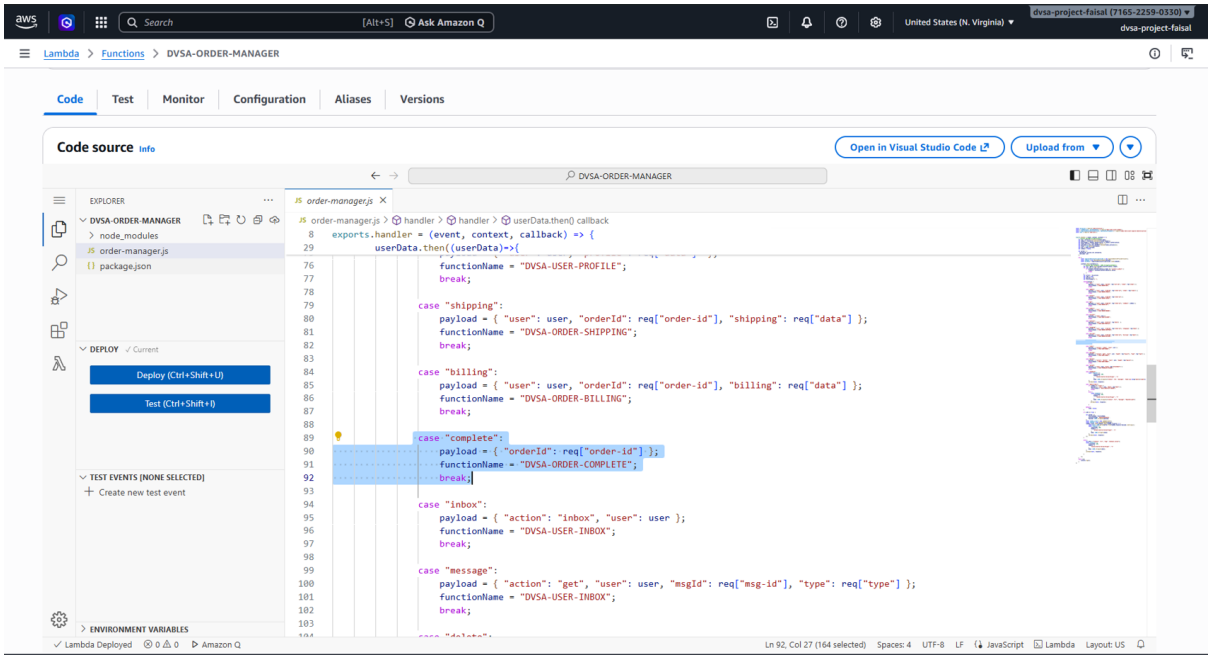

Figure 1. Public *complete* action in *DVSA-ORDER-MANAGER* routes to *DVSA-ORDER-COMPLETE*.

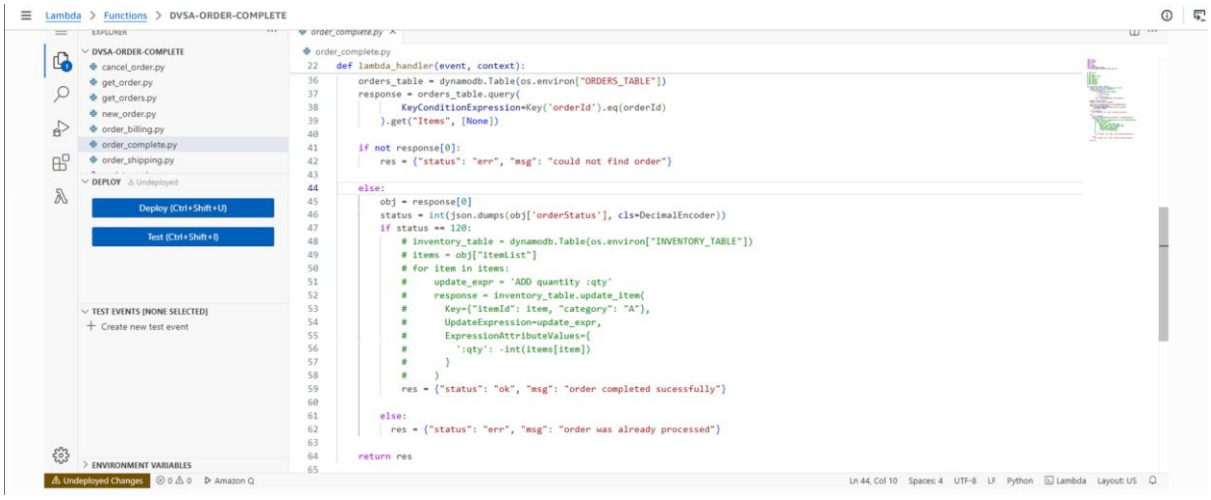

Figure 2. *DVSA-ORDER-COMPLETE* only succeeds when the order status is *120*; otherwise it returns an already-processed style error.

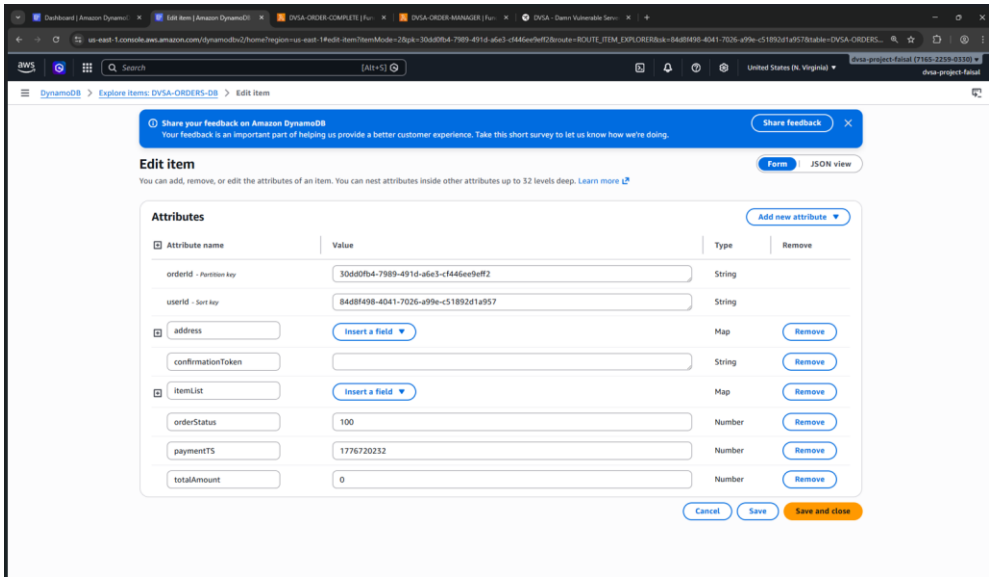

Figure 3. Unfinished second order in DynamoDB before normal billing, showing *orderStatus = 100*.

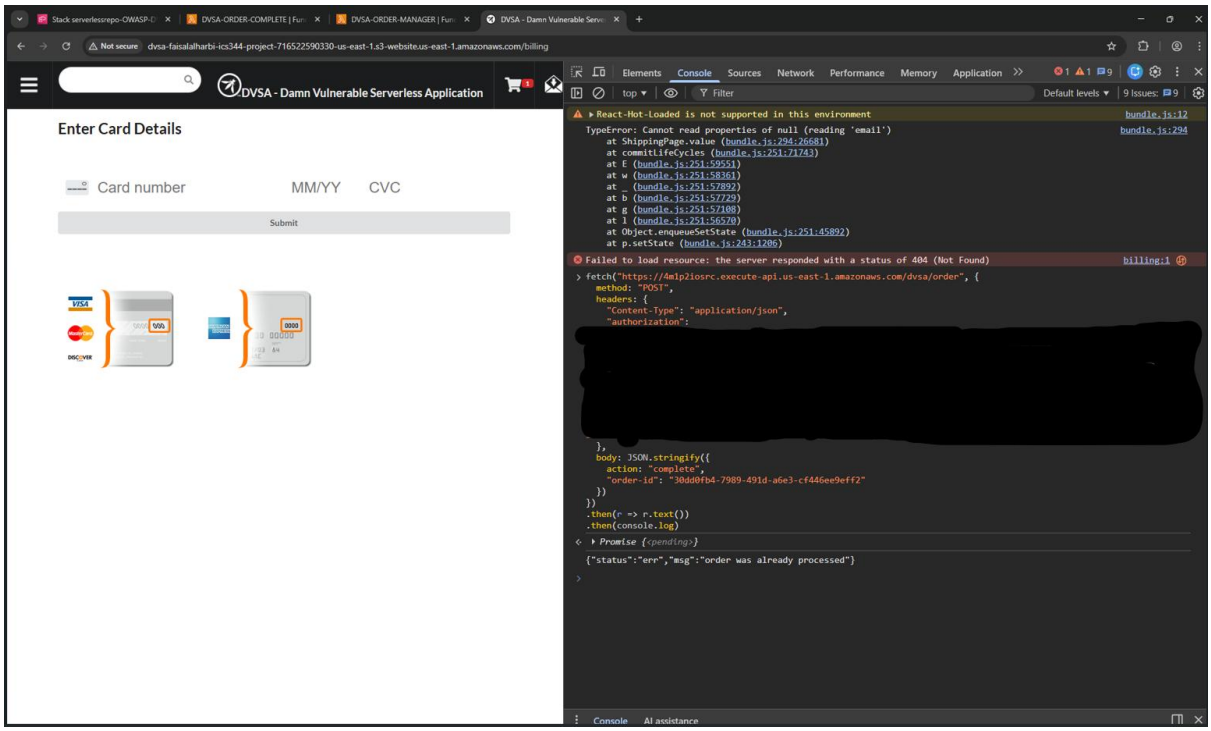

Figure 4. Direct call to the public *complete* action returned an error instead of advancing the unfinished order.

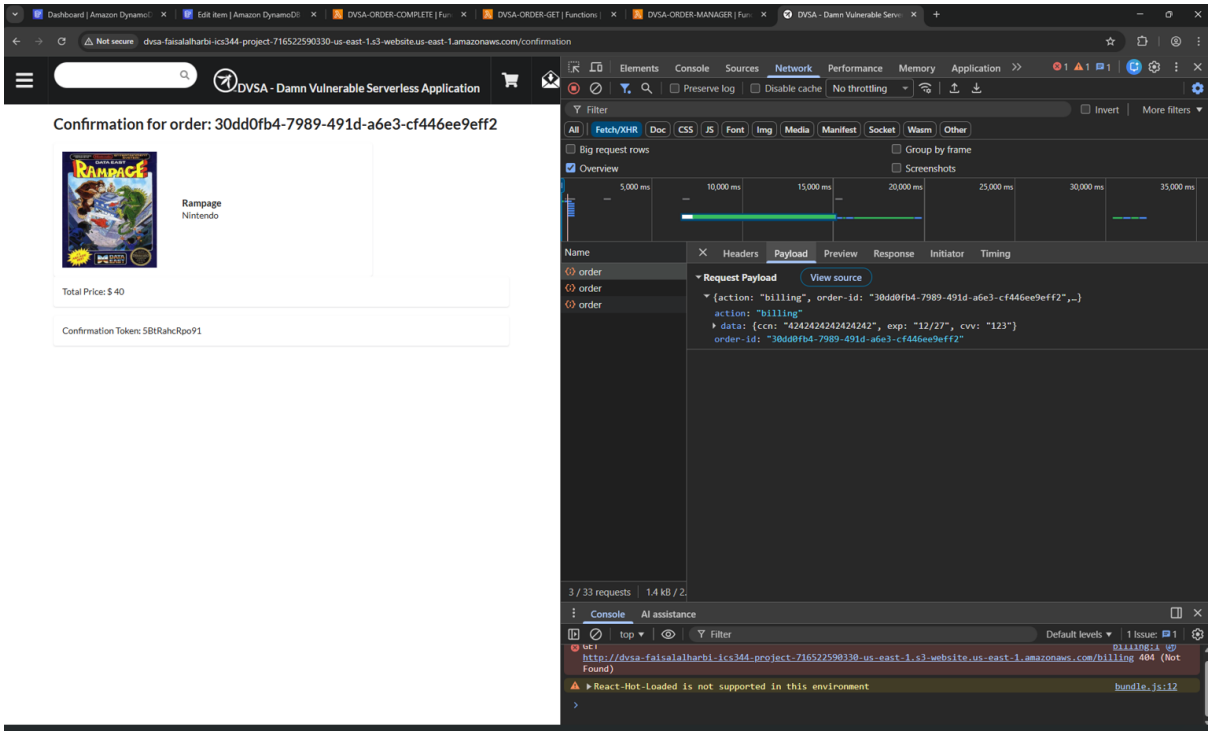

Figure 5. Normal billing request captured from the frontend order workflow.

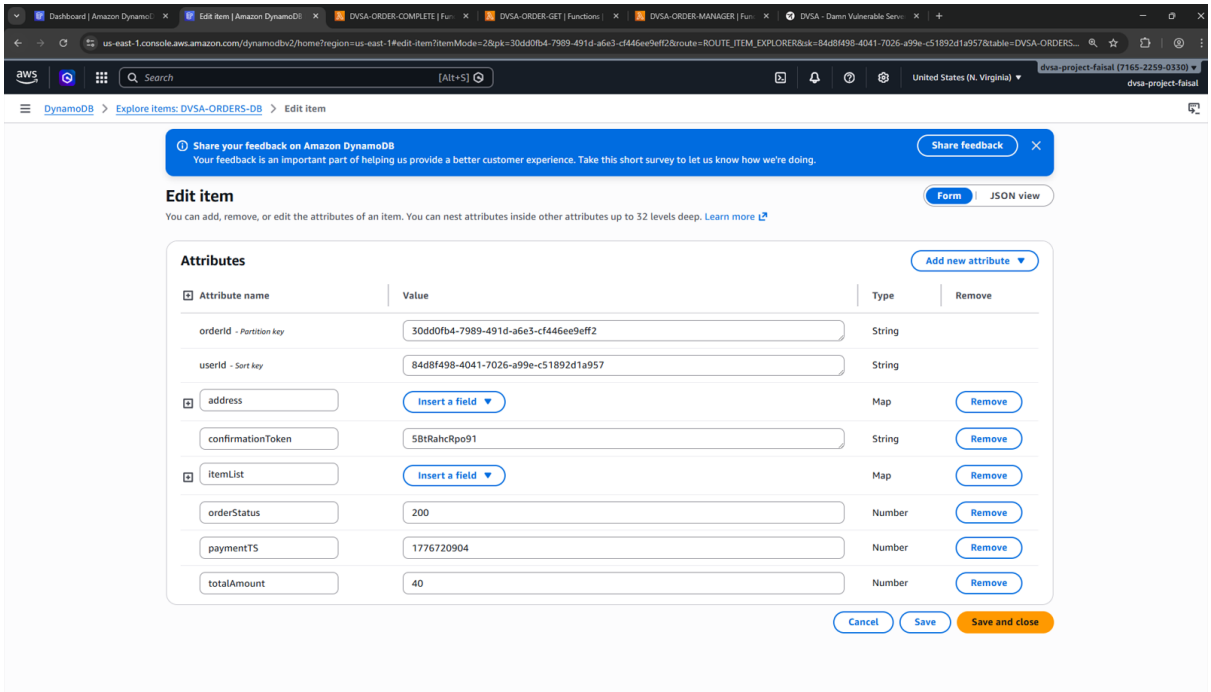

Figure 6. DynamoDB record after legitimate billing, showing processed workflow data.

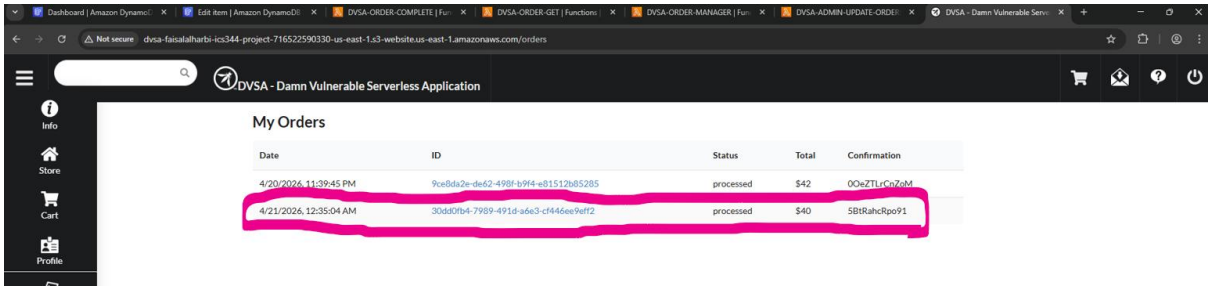

Figure 7. Final order visible in the DVSA interface after normal billing, shown as processed.

Part 6 — Fix Strategy / Probable Mitigation

A proper mitigation for this lesson should focus on removing or tightly restricting sensitive order-state actions from the public API path and enforcing strict server-side workflow validation. In particular, the public complete action exposed by DVSA-ORDER-MANAGER should not be reachable by an unprivileged frontend user unless the backend explicitly verifies that the caller is authorized to perform that transition. Sensitive order-management functions should validate both the caller identity and the current order state before changing any record in DynamoDB. In addition, state transitions should be enforced using a strict server-side allowlist so that an order can move only through the intended sequence of workflow states. Finally, the backend code should be made consistent by fixing the isAdmin handling in DVSA-ORDER-GET, since inconsistent authorization-related behavior can interfere with both security testing and safe application behavior. Although I did not reproduce the intended exploit path in this environment, these changes would reduce the attack surface and strengthen the access-control boundary around sensitive order operations.

Part 7 — Code / Config Changes

No code or configuration changes were applied in this deployment during the Lesson 5 investigation because the intended unauthorized order-state change was not reproducible in my environment. However, the code review identified the main backend locations where a corrective hardening change should be applied if this workflow is to be secured more strictly. First, in *DVSA-ORDER-MANAGER*, the public *complete* action should be removed or explicitly blocked for normal frontend users, since it currently routes directly to *DVSA-ORDER-COMPLETE* using only an *orderId*. Second, *DVSA-ORDER-COMPLETE* should enforce a stricter trust boundary by verifying that the call originates from an authorized internal workflow and by rejecting any attempt to use the function as a public order-state action. Third, *DVSA-ORDER-GET* should be corrected so that the *isAdmin* value is handled consistently without causing runtime errors when the value is not a string. Finally, any administrative update path such as *DVSA-ADMIN-UPDATE-ORDERS* should enforce strict authorization and allow only valid server-side order-state transitions. These changes were identified as recommended hardening points, but they were not applied as part of this lesson because the target exploit was not successfully reproduced.

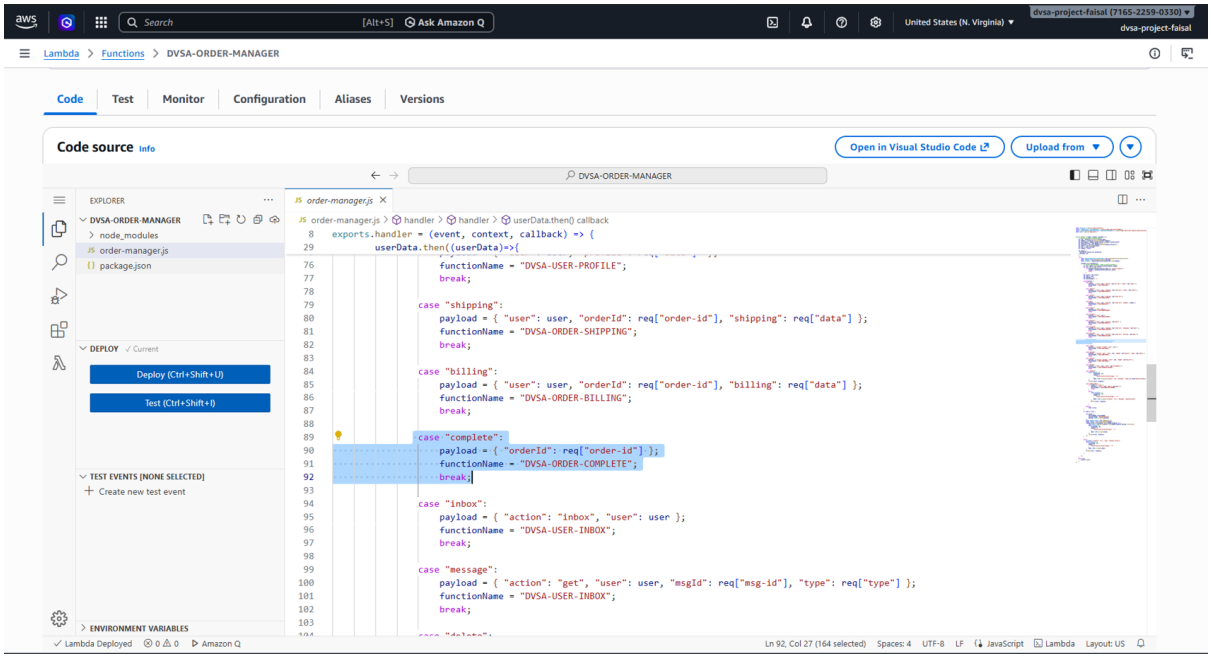

Figure 8. Public *complete* action in *DVSA-ORDER-MANAGER*, identified as a recommended hardening point.

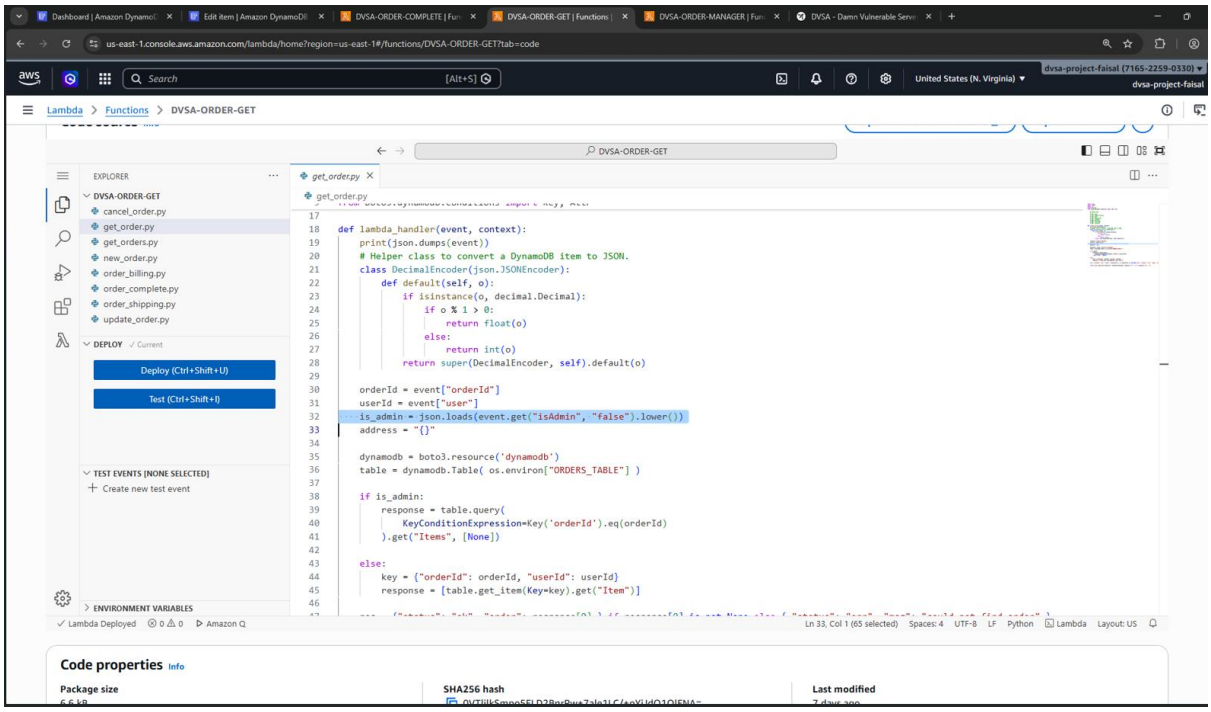

Figure 9. Inconsistent *isAdmin* handling in *DVSA-ORDER-GET*, identified as a backend reliability and authorization-related hardening issue.

Part 8 — Verification After Fix

No actual code or configuration fix was applied during this Lesson 5 investigation, because the intended unauthorized order-state change was not reproducible in my deployed environment. Therefore, no true post-fix verification was performed. The verification completed in this lesson was limited to exploit testing and normal workflow comparison. In that testing, the direct call to the public complete action did not improperly advance the unfinished order and instead

returned an error, while the same order moved to the expected later state only after the legitimate billing workflow was completed. If a hardening fix were later applied, proper post-fix verification would require repeating the same direct complete request and confirming that it is explicitly rejected by the backend, while also confirming that normal shipping, billing, and order-processing behavior still works correctly for a legitimate user.

The screenshots and API responses shown in Part 5 serve as the observed verification evidence for the tested deployment.

Part 9 — Structured Operation and Security Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Broken Access Control	Only authorized workflow steps may change order state; a normal user must not trigger sensitive completion logic outside billing.	DVSA browser workflow, /dvsa/order requests, DVSA-ORDER-MANAGER, DVSA-ORDER-COMPLETE, DVSA-ORDER-BILLING, DVSA-ORDER-GET, DynamoDB records, Orders page.	Normal billing moved the order through the expected workflow, and after legitimate checkout the order appeared as processed with a confirmation token and updated record in DynamoDB.	A direct public complete request using a normal user token returned an error and did not improperly advance the unfinished order; a separate DVSA-ORDER-GET bug caused runtime failure during order retrieval.

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Broken Access Control	The public API exposes a sensitive complete action without a clear authorization check in that routing path, which creates a weak backend trust boundary even though the final exploit outcome was not reproduced in this deployment.	Security-relevant design weakness / incomplete access-control enforcement	No fix applied in this lesson. Recommended hardening points were identified in DVSA-ORDER-MANAGER, DVSA-ORDER-COMPLETE, and DVSA-ORDER-GET.	No post-fix verification performed because no patch was applied. Observed testing showed the direct complete request failed and normal billing still worked correctly.	Not measured

Part 10 — Takeaway / Lessons Learned

This lesson showed that even when the exact exploit path is not reproducible in a specific DVSA deployment, exposed sensitive order actions and inconsistent backend checks are still important security findings. The main security lesson is that workflow-sensitive operations, such as order completion or other state-changing actions, should never be exposed through a public API path unless the backend enforces strict authorization, ownership checks, and valid server-side state transitions. In a serverless application, these boundaries matter even more because a single public action may route directly into internal Lambda logic. My testing did not confirm the exact Lesson 5 exploit in this environment, but it did show that weak backend trust

boundaries and inconsistent validation can make sensitive workflows harder to secure and easier to misinterpret during both normal use and security testing.